

Pose & Resample Reference

Every command to install, build, find a projection center-of-rotation pose, resample against it, and validate the result — across all three interchangeable backends.

● **openc1** • PyOpenCL, GPU

● **cpu** • NumPy, no GPU

● **cpp** • forward_search/ binaries

01 Setup

Run once. The second block is Linux-only, and only needed for the cpp backend.

Install the Python package

Editable install; registers the `projection-center` command.

```
cd projection-center && python3 -m pip install -e . && cd ..
```

Build the C++/OpenCL binaries (Linux only)

Configures, then compiles, via meson and ninja.

```
cd forward_search && meson setup builddir && meson compile -C builddir && cd ..
```

List visible OpenCL devices

Queries every OpenCL platform and prints what it finds.

```
projection-center devices
```

02 Datasets

Two datasets currently live under `data/` (gitignored, populate locally). Commands below spell out every parameter explicitly — nothing left to a default.

`data/projs_change.hdf5` · the real dataset

projections	180
detector	1024×1024
SDD / SOD	7.0 / 5.0
pixel size	0.0039

```
projection-center pipeline --backend opengl --data data/projs_change.hdf5 --xshift 40.0
--alpha 10.0 --beta 10.0 --xshift-step 1.0 --alpha-step 1.0 --beta-step 1.0 --downsample
1 --batch-size 16 --output-pose runs/real_run/pose.json --output-data
runs/real_run/resampled.hdf5
```

`data/proj_shepplogan128.hdf5` · Shepp–Logan phantom

projections	500
detector	128×128
SDD / SOD	1085.6 / 416.5
pixel size	1.731

```
projection-center pipeline --backend opengl --data data/proj_shepplogan128.hdf5 --xshift
40.0 --alpha 10.0 --beta 10.0 --xshift-step 1.0 --alpha-step 1.0 --beta-step 1.0 --
downsample 1 --batch-size 16 --output-pose runs/shepp_run/pose.json --output-data
runs/shepp_run/resampled.hdf5
```

▲ **Shepp–Logan caveat:** the found `xshift` is weakly determined on this dataset — a property of the phantom's own symmetry, not a bug (see Limitations).

Swap `--backend` for `cpu/cpp` exactly as in the command matrix below — every flag here works identically on either dataset, on any backend.

03 The command matrix

Every subcommand runs on every backend — nine combinations in total. Swap --backend, everything else stays the same.

search

Find the pose

opengl

Grid-searches the pose on the GPU with the package's own PyOpenCL kernel.

```
projection-center search --backend opengl --data data/projs_change.hdf5 --  
output-pose runs/my_run/pose.json
```

Find the pose

cpu

Same grid search, vectorized in NumPy, no GPU involved.

```
projection-center search --backend cpu --data data/projs_change.hdf5 --  
output-pose runs/my_run/pose.json
```

Find the pose

cpp

Subprocesses into the compiled forward_search binary and reads its result back.

```
projection-center search --backend cpp --data data/projs_change.hdf5 --  
output-pose runs/my_run/pose.json --cpp-mode buffer
```

Apply a pose

opengl

Ray-casts every output pixel back into the original image, on the GPU.

```
projection-center resample --backend opengl --data data/projs_change.hdf5 -  
-pose runs/my_run/pose.json --output-data runs/my_run/resampled.hdf5
```

Apply a pose

cpu

Same ray-casting, vectorized in NumPy.

```
projection-center resample --backend cpu --data data/projs_change.hdf5 --  
pose runs/my_run/pose.json --output-data runs/my_run/resampled.hdf5
```

resample

pipeline

Apply a pose

cpp

Subprocesses into the compiled forward_search_resample binary.

```
projection-center resample --backend cpp --data data/projs_change.hdf5 --  
pose runs/my_run/pose.json --output-data runs/my_run/resampled.hdf5 --cpp-  
mode buffer
```

Search, then resample

opencl

Runs the two commands above back to back, same backend for both.

```
projection-center pipeline --backend opencl --data data/projs_change.hdf5 -  
-output-pose runs/my_run/pose.json --output-data runs/my_run/resampled.hdf5
```

Search, then resample

cpu

Same, entirely on CPU.

```
projection-center pipeline --backend cpu --data data/projs_change.hdf5 --  
output-pose runs/my_run/pose.json --output-data runs/my_run/resampled.hdf5
```

Search, then resample

cpp

Same, via the two compiled binaries in turn.

```
projection-center pipeline --backend cpp --data data/projs_change.hdf5 --  
output-pose runs/my_run/pose.json --output-data runs/my_run/resampled.hdf5  
--cpp-mode buffer
```

- ✓ **Confirmed working, all nine.** Real dataset, all three pipeline runs: identical winning pose (xshift=-20mm, alpha=-5°, beta=4°) on every backend, resampled pixels agreeing to ~1e-6 mean difference across every pair. Timing: opencl 27s, cpp 11s, cpu 1m20s.

04 Optional flags

Same meaning on every backend unless noted. Defaults shown.

FLAG	DEFAULT	APPLIES TO
--xshift / --alpha / --beta	40.0 / 10.0 / 10.0	search, pipeline · $\pm$ range, mm / deg
--xshift-step / --alpha-step / --beta-step	1.0 / 1.0 / 1.0	search, pipeline
--downsample	1	resample, pipeline
--batch-size	16	resample, pipeline · GPU batching only
--cpp-mode	buffer	cpp only · image buffer

cpp does not support non-default --sample-count / --sample-angle-range or --platform-index / --device-index — forward_search/'s CLI hardcodes these.

05 Untouched Python reference

Serial, slow, must stay unmodified. Everything above validates against this.

Forward search · ~10 minutes · writes ./real_cb_pose.json

Plain Python loop over every candidate pose, one at a time.

```
python3 reference/Topic_3_forwardsearching.py --data data/projs_change.hdf5
```

Resampling · reads ./real_cb_pose.json

Plain Python loop over every output pixel, one at a time.

```
python3 reference/Topic_3_resampling.py --data data/projs_change.hdf5
```

06 Validating a pose against the reference

Three ways to run it, in increasing rigor:

Against an existing reference pose file

Just diffs two already-computed poses; runs nothing new.

```
python3 validate_forward_search.py --gpu runs/my_run/pose.json --ref
runs/reference_baseline/pose.json
```

Run the raw reference now (~10 min)

Runs the untouched reference script itself, bug included, as the baseline.

```
python3 validate_forward_search.py --gpu runs/my_run/pose.json --run-ref --data
data/projs_change.hdf5
```

Recommended · bug-fixed reference, in-process only

Same run, with the reference's bug patched in memory only, file untouched.

```
python3 validate_forward_search.py --gpu runs/my_run/pose.json --run-ref --fix-ref --
data data/projs_change.hdf5
```

① **Why --fix-ref:** the raw reference has a known bug (see Limitations) that can shift its own optimum by up to 1.8° in beta. The corrected run patches it in memory only — the reference file is never touched.

07 Benchmarking all backends

Runs the raw reference, then `opencl/cpu/cpp`, one after another, and prints a single timing + pose table. Saves every output under `runs/benchmark_all/<name>/`.

Full run · includes the raw reference (~10–27 min for that step)

Reference (search only) + `opencl/cpu/cpp` (search + resample), timed and compared.

```
python3 benchmark_backends.py --data data/projs_change.hdf5
```

Skip the slow part

Just `opencl/cpu/cpp` against each other — seconds, not minutes.

```
python3 benchmark_backends.py --data data/projs_change.hdf5 --skip-reference
```

- ① The reference row times **search only** (its own resample script can't complete off the lab network); the three backend rows time **search + resample**. Not a clean apples-to-apples speedup ratio — the output table labels each row so it isn't misread as one.

08 Comparing backends yourself

Differs two already-produced `resampled.hdf5` files pixel-by-pixel — e.g. after running the benchmark above, compare `runs/benchmark_all/opencv/resampled.hdf5` against `runs/benchmark_all/cpu/resampled.hdf5`.

Quick • first 5 projections

Loads just 5 projections from each file and reports the pixel-wise difference.

```
python3 -c "  
import h5py, numpy as np  
a = h5py.File('a/resampled.hdf5')['Projection'][:5]  
b = h5py.File('b/resampled.hdf5')['Projection'][:5]  
d = np.abs(a.astype(float)-b.astype(float))  
print(d.max(), d.mean())"
```

Full • all 180 projections

Same idea, over the whole dataset instead of a sample.

```
python3 -c "  
import h5py, numpy as np  
a = h5py.File('a/resampled.hdf5')['Projection'][:]  
b = h5py.File('b/resampled.hdf5')['Projection'][:]  
d = np.abs(a.astype(float)-b.astype(float))  
print(d.max(), d.mean(), np.median(d))"
```

09 Screenshots

Nothing above renders an image on its own — output is raw float32 HDF5. Pull a slice, save as PNG. The two commands below use the same projection index against the original and the corrected file, so the two PNGs are directly comparable.

Original (before correction) → PNG · needs pillow

Same idea, pointed at the raw input instead of the resampled output.

```
python3 -c "  
import h5py, numpy as np  
from PIL import Image  
p = h5py.File('data/projs_change.hdf5')['Projection'][0]  
n = (p - p.min()) / (p.max() - p.min())  
Image.fromarray((n*255).astype(np.uint8)).save('projection_0_original.png')"
```

Resampled (corrected) → PNG · needs pillow

Reads one projection, scales it to 0–255, saves it as an image.

```
python3 -c "  
import h5py, numpy as np  
from PIL import Image  
p = h5py.File('runs/my_run/resampled.hdf5')['Projection'][0]  
n = (p - p.min()) / (p.max() - p.min())  
Image.fromarray((n*255).astype(np.uint8)).save('projection_0.png')"
```

10 Limitations

- ▲ **cpp is Linux-only** as built here (meson, Boost, HDF5 C++ via system packages). On Windows use `--backend opengl` or `--backend cpu`; both are pure Python.
- ▲ `--mode image` **crashes** on this dev machine's Intel iGPU driver. `--mode buffer` is what's validated.
- ▲ **Reference has a known bug:** a ray that misses the detector can silently reuse the previous ray's value. Use `--fix-ref` when validating.
- ▲ **Topic_3_resampling.py** writes to a hardcoded lab-network path; it fails off that network. Use `projection-center resample` instead.
- ▲ On the small Shepp–Logan dataset, found `xshift` is weakly determined — a property of the dataset, not a bug.
- ▲ KDevelop project packaging is not done yet.